

5. 합성곱 신경망 I

5.3 전이 학습

일반적으로 합성곱 신경망 기반의 딥러닝 모델을 제대로 훈련시키려면 많은 양의 데이터가 필요함 → 충분히 큰 데이터셋을 얻기 위해서는 많은 돈과 시간이 필요하기 때문에 큰 데이터셋 확보가 쉽지 않음

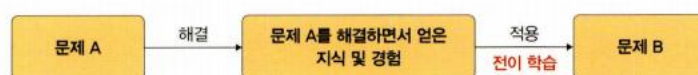
전이 학습(transfer learning)

- ImageNet처럼 아주 큰 데이터셋을 써서 훈련된 모델의 **가중치**를 가져옴
- 가져온 가중치를 해결하려는 과제에 맞게 보정하여 사용
- 아주 큰 데이터셋을 사용하여 훈련된 모델을 **사전 훈련된 모델(네트워크)**이라고 함
- 비교적 적은 수의 데이터를 가지고도 원하는 과제를 해결할 수 있음

전이 학습을 위한 방법

- **특성 추출 기법**
- **미세 조정 기법**

▼ 그림 5-29 전이 학습



5.3.1 특성 추출 기법

특성 추출(feature extractor)

ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 **마지막의 완전연결층 부분만 새로 만드는 방법**

- 학습할 때는 마지막 완전연결층(이미지 카테고리를 결정하는 부분)만 학습
- 나머지 계층들은 학습되지 않도록 함

특성 추출을 이용한 이미지 분류는 두 부분으로 구성

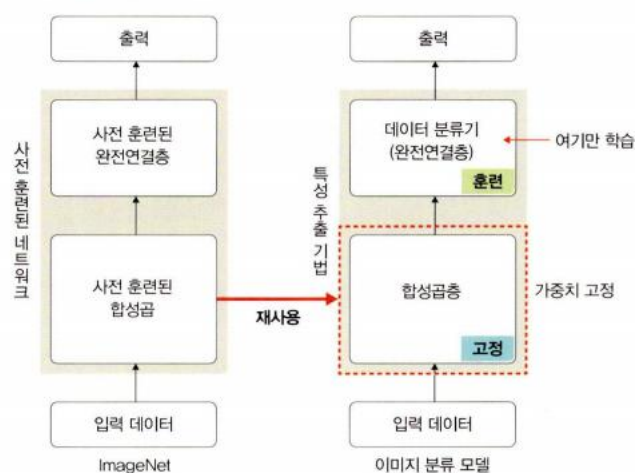
- **합성곱층** : 합성곱층과 풀링층으로 구성
- **데이터 분류기(완전연결층)** : 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스를 분류

사전 훈련된 네트워크의 **합성곱층은 가중치를 고정**한 상태로 새로운 데이터를 통과시킴
→ 그 출력값을 데이터 분류기에서 **훈련**

사용 가능한 이미지 분류 모델

- Xception
- Inception V3
- ResNet50
- VGG16
- VGG19
- MobileNet

♥ 그림 5-30 특성 추출 기법



OpenCV

특성 추출 예제를 진행하기 위해 필요한 라이브러리 중 하나

- Open Source Computer Vision Library의 약어
- 오픈 소스 컴퓨터 비전 라이브러리
- OpenCV-Python 라이브러리를 설치하면 파이썬에서도 사용 가능

이미지 데이터 전처리

torchvision.transforms

→ 이미지 데이터를 변환하여 모델(네트워크)의 입력으로 사용할 수 있도록 변환

transforms.Compose()에서 사용되는 전처리

Resize

- 이미지의 크기를 조정

RandomResizedCrop

- 이미지를 랜덤한 크기 및 비율로 자름
- Resize와 RandomResizedCrop 모두 이미지를 자르는 데 사용하지만 용도가 다름
- Resize : 합성곱층을 통과하기 위해 이미지 크기를 조정하는 **전처리 과정**
- RandomResizedCrop : **데이터 확장** 용도로 사용
- 이미지를 랜덤한 비율로 자른 후 데이터 크기를 조정

RandomHorizontalFlip

- 이미지를 랜덤하게 수평으로 뒤집음

ToTensor

- 이미지 데이터를 텐서로 변환

ImageFolder

datasets.ImageFolder

데이터로더가 데이터를 불러올 대상(혹은 경로)과 방법(transform, 혹은 전처리)을 정의

주요 파라미터

- 첫 번째 파라미터 : 불러올 데이터가 위치한 경로
- transform : 이미지 데이터에 대한 전처리

DataLoader

ImageFolder를 데이터로더에 할당하여 데이터를 불러옴

주요 파라미터

- 첫 번째 파라미터 : 데이터셋 지정

- `batch_size` : 한 번에 불러올 데이터양을 결정하는 배치 크기
- `num_workers` : 데이터를 불러올 때 하위 프로세스를 몇 개 사용할지 설정
 - 너무 많은 하위 프로세스를 설정하면 오류가 발생하거나 메모리 부족 현상이 발생할 수 있음
- `shuffle`
 - 데이터를 무작위로 섞을지 지정
 - `shuffle=True`이면 데이터를 무작위로 섞어서 랜덤으로 불러옴

RandomResizedCrop

교재에서는 `RandomResizedCrop`을 이용해 데이터 확장 방법을 추가로 설명

주요 파라미터

- `size`
 - 출력할 크기
- `scale`
 - 면적 비율을 **0.1~1(10~100%)** 범위에서 무작위로 자름
- `ratio`
 - 면적의 너비와 높이 비율을 **0.5~2** 범위에서 무작위로 조절

즉, `ratio`를 통해 너비와 비율을 조절

→ `scale`을 통해 자름

→ 최종적으로 출력할 `size`로 조정

♥ 그림 5-32 `RandomResizedCrop`이 적용된 이미지



iter() / next()

반복자(iterator), for 구문과 같은 효과를 사용하려면 iter()와 next()가 필요

- iter() : 전달된 데이터의 반복자를 꺼내 반환
- next() : 반복자가 다음에 출력해야 할 요소를 반환

train_loader가 반복자이므로

→ train_loader에서 samples와 labels 값을 순차적으로 꺼내 저장

즉, train_loader에서 데이터를 하나씩 꺼내 오는 과정

np.transpose()

np.transpose

→ 행과 열을 바꿈으로써 **행렬의 차원을 변경**

행렬의 차원을 변경하는 이유 중 하나는 **행렬의 내적 연산**

A 행렬과 B 행렬을 내적하려면 → A 행렬의 행과 B 행렬의 열의 수가 같아야 함

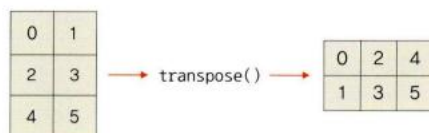
행과 열의 수가 달라 내적이 불가능하다면 np.transpose()나 np.reshape() 등을 이용하여 차원을 조정

ex. 교재의 이미지 데이터 형태

torch.Size([32, 3, 224, 224]) 그대로 사용하면 Invalid shape (3, 224, 224) for image data 오류 발생

-> 따라서 np.transpose(samples[i].numpy(), (1,2,0))을 이용하여 (224, 224, 3) 형태로 변환 후 사용

▼ 그림 5-33 transpose()



ResNet18

데이터 준비 후 사전 훈련된 **ResNet18 모델**을 사용

pretrained=True → 사전 학습된 가중치를 사용한다는 의미

ResNet18

- 50개의 계층으로 구성된 합성곱 신경망
- ImageNet 데이터베이스의 100만 개가 넘는 영상을 이용하여 훈련된 신경망
- 전이 학습에 사용되도록 사전 훈련된 모델 제공
- 입력 자체가 매우 크고 충분한 메모리(RAM)가 없으면 학습 속도가 느릴 수 있다는 단점

파이토치는 모델을

- 무작위의 가중치로 생성하거나
- pretrained=True를 이용해 **사전 학습된 가중치**를 사용할 수 있음.

사전 훈련된 모델의 파라미터 고정

특성 추출에서는 ResNet18의 **합성곱층은 사용하지만 파라미터는 학습하지 않도록 고정**

`requires_grad = False`

→ 역전파 중 파라미터에 대한 변화를 계산할 필요가 없음을 의미

→ 모델의 일부를 고정하고 나머지만 학습하고자 할 때 사용

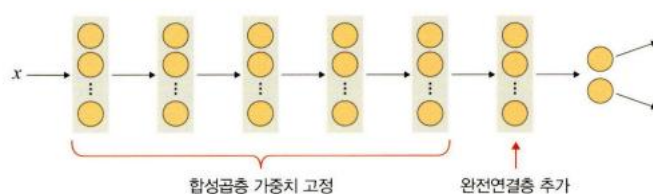
여기서 고정되는 모델의 일부

- 합성곱층(convolutional layer)
- 풀링층(pooling)

그 후 ResNet18의 마지막 부분에 **완전연결층**을 추가

추가된 완전연결층은 개와 고양이 **두 클래스**를 분류하는 용도로 사용

▼ 그림 5-36 ResNet18에 완전연결층 추가



ResNet18 파라미터

추가된 완전연결층에서는

- weight

- bias

파라미터 사용

특성 추출에서는

- 기존 합성곱층의 가중치 : 고정
- 새로 추가한 완전연결층 : 학습

하도록 설정

완전연결층에서 학습을 통해 얻어지는 파라미터를 옵티마이저에 전달하여 최종 모델 학습에 사용

교재에서 모델 훈련을 위해 전달하는 파라미터

- 모델
- 학습 데이터
- 손실 함수
- 옵티마이저
- 장치(CPU 혹은 GPU)

모델 훈련 결과와 테스트 데이터

훈련 결과 약 **93%의 정확도**

→ 훈련 데이터에서는 학습이 잘되었다고 할 수 있음

→ 실제 성능을 확인하기 위해 **테스트 데이터**를 이용하여 모델 정확도를 측정

테스트 데이터 전처리

- Resize(224)
- CenterCrop(224)
- ToTensor()

테스트 데이터 평가

glob

현재 디렉터리에서 원하는 파일들만 추출하여 가져올 때 사용

교재에서는

.pth 확장자를 가진 파일을 가져옴

.pth 파일

→ 훈련 데이터로 모델을 훈련시킬 때 생성된 파일

torch.max

주어진 텐서 배열의 **최댓값이 들어 있는 index**를 반환

preds.eq(labels)

preds 배열과 labels가 일치하는지 검사

.sum()

→ 모델의 예측 결과와 정답(레이블)이 일치하는 것들의 개수를 숫자로 출력

정확도와 오차 그래프

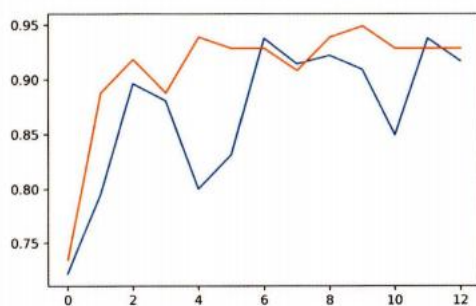
훈련과 테스트 데이터의 에포크별 정확도를 확인

- 에포크가 진행될수록 정확도가 높아짐
- 훈련 데이터와 테스트 데이터 모두 100%에 가까워짐

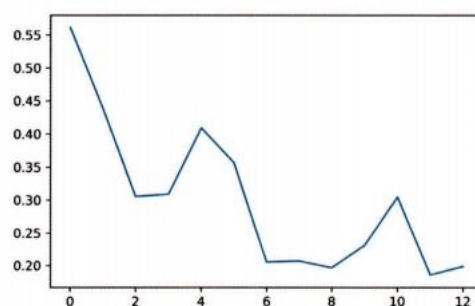
훈련 데이터의 오차

- 에포크가 진행될수록 오차가 낮아짐
- 이를 통해 학습이 잘되고 있음을 확인

▼ 그림 5-37 훈련과 테스트 데이터에 대한 정확도



▼ 그림 5-38 훈련 데이터에 대한 오차



tensor.clone() / detach()

예측한 이미지를 출력하기 위한 전처리 과정에서 사용

tensor.clone()

- 기존 텐서의 내용을 복사한 새로운 텐서를 생성
- 메모리를 새롭게 할당
- 계산 그래프는 계속 상주

tensor.detach()

- 기존 텐서와 메모리를 공유
- 계산 그래프는 상주하지 않음

tensor.clone().detach()

- 기존 텐서를 복사한 새로운 텐서를 생성
- 기울기에 영향을 주지 않음
- 메모리를 새롭게 할당
- 계산 그래프는 상주하지 않음

계산 그래프(computational graph)

계산 과정을 그래프로 나타낸 것

구성

- **노드(node)**
- 노드를 연결하는 선인 **에지(edge)**

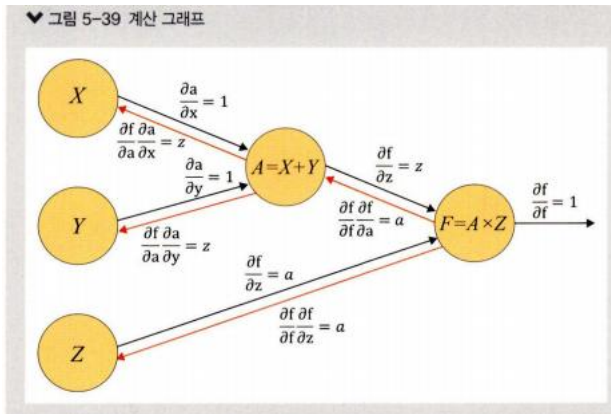
계산 그래프를 사용하는 이유

1. 국소적 계산이 가능

- 일부 값이 변경되어도 전체를 다시 계산하지 않고 필요한 연산만 계산 가능

2. 역전파를 통한 미분 계산이 편리

- 연쇄 법칙(chain rule)을 이용하여 빠르고 간편하게 미분 계산 가능

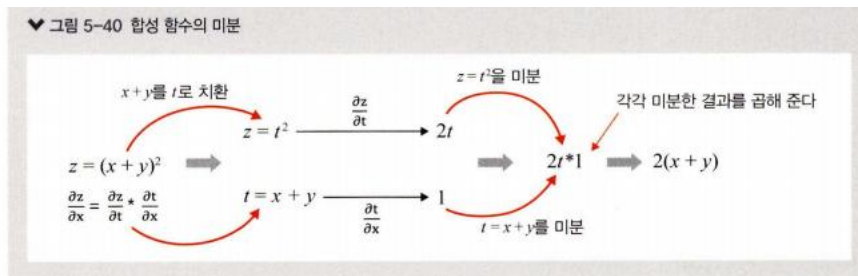


연쇄 법칙

두 개 이상의 함수가 결합된 함수, 즉 합성 함수의 미분법을 연쇄 법칙 또는 체인룰(chain rule)이라고 함

합성 함수를 미분하고자 할 때

→ 각 합성 함수의 미분 결과를 곱한 결과로 미분값을 구함



clip()

입력 값이 주어진 범위를 벗어날 때 입력 값을 특정 범위로 제한할 때 사용

예) image.clip(0, 1) → 이미지 데이터를 **0과 1 사이의 값으로 제한**

add_subplot

한 화면에 여러 개의 이미지를 담기 위해 사용

교재 예에서 사용되는 파라미터

- 첫 번째 파라미터 : 행의 수
- 두 번째 파라미터 : 열의 수
- 세 번째 파라미터 : 인덱스
 - 행과 열을 기준으로 순차적으로 이미지를 출력

- 네 번째 파라미터
 - xticks, yticks
 - 틱(tick)을 삭제

classes[preds[idx].item()]

→ 예측값이 0이면 고양이, 1이면 개로 출력

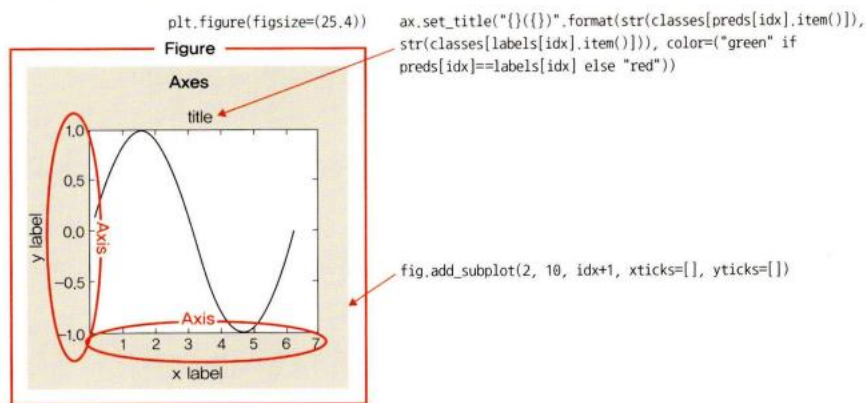
subplots_adjust

→ Figure 안에서 서브플롯의 위치를 조정

→ left, bottom, right, top으로 위치 조정

→ hspace, wspace로 서브플롯 사이의 간격 조정

♥ 그림 5-42 서브플롯



개와 고양이 예측 결과

교재의 예측 결과

- 초록색 : 개와 고양이를 정확하게 예측
- 빨간색 : 예측이 잘못됨

예측이 정확하지 않은 것을 확인할 수 있음

→ 훈련 데이터를 더 늘리고

→ 에포크 횟수도 늘려 보면 더 좋은 결과를 얻을 수 있음

♥ 그림 5-43 개와 고양이에 대한 예측 결과



5.3.2 미세 조정 기법

미세 조정(fine-tuning)

특성 추출 기법에서 더 나아가 **사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식**

특성 추출은 목표 특성을 잘 추출했다는 전제하에 좋은 성능을 낼 수 있음

하지만 특성이 잘못 추출되었다면

예) ImageNet 데이터셋의 이미지 특징과 전자상거래 물품의 이미지 특징이 다른 경우

- 새로운 이미지 데이터를 사용
- 네트워크의 가중치를 업데이트
- 특성을 다시 추출할 수 있음

즉,

- 사전 학습된 모델을 목적에 맞게 재학습시키거나
- 학습된 가중치의 일부를 재학습

하는 방식

미세 조정 → 사전 훈련된 네트워크를 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터를 조정. 많은 연산량이 필요하므로 **CPU보다는 GPU 사용 권장**

데이터셋 크기와 사전 훈련된 모델의 유사성에 따른 미세 조정 전략

데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우

- **모델 전체를 재학습**
- 데이터셋 크기가 크기 때문에 재학습시키는 것이 좋은 전략

데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우

- 합성곱층의 뒷부분(완전연결층과 가까운 부분)과 데이터 분류기를 학습
- 전체를 학습시키기보다 강한 특징이 나타나는 합성곱층 뒷부분과 데이터 분류기만 새로 학습해도 최적의 성능을 낼 수 있음

데이터셋이 작고 사전 훈련된 모델과 유사성이 작을 경우

- 합성곱층의 일부분과 데이터 분류기를 학습

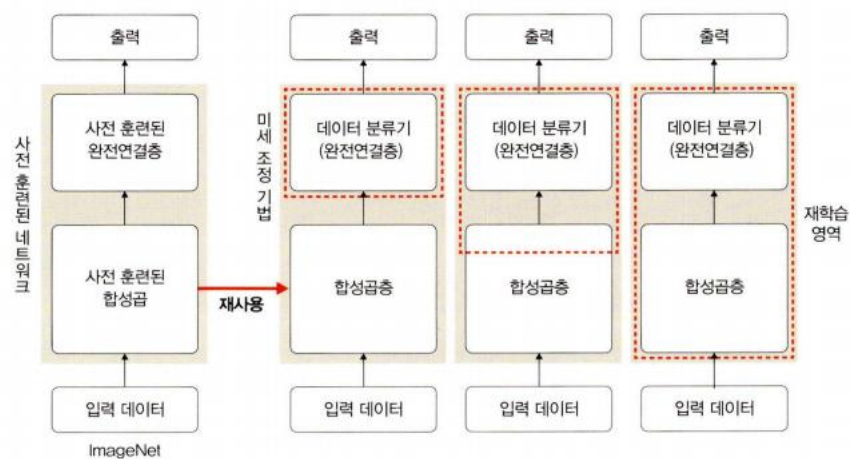
- 데이터가 적기 때문에 일부 계층에 미세 조정을 적용해도 효과가 없을 수 있음
- 합성곱층 중 어디까지 새로 학습시켜야 할지 적당히 설정해야 함

데이터셋이 작고 사전 훈련된 모델과 유사성이 클 경우

- 데이터 분류기만 학습
- 많은 계층에 미세 조정을 적용하면 과적합이 발생할 수 있음
- 최종 데이터 분류기인 완전연결층에 대해서만 미세 조정 적용

미세 조정은 파라미터 업데이트 과정에서 파라미터에 큰 변화를 주면 **과적합** 문제가 발생할 수 있으므로 **정교하고 미세한 파라미터 업데이트**가 필요

♥ 그림 5-44 미세 조정 기법



5.4 설명 가능한 CNN

설명 가능한 CNN(explainable CNN)

딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술

CNN은 블랙박스(blackbox)와 같이 내부에서 어떻게 동작하는지 설명하기 어려움

→ CNN으로 얻은 결과를 신뢰하기 어려울 수 있음

→ 이를 해결하려면 CNN의 처리 과정을 시각화할 필요가 있음

CNN을 구성하는 각 중간 계층부터 최종 분류기까지

입력된 이미지에서 특성이 어떻게 추출되고 학습되는지를 시각적으로 설명할 수 있어야 결과에 대한 신뢰성을 얻을 수 있음

CNN 시각화 방법

- 필터에 대한 시각화
- 특성 맵에 대한 시각화

교재에서는 **특성 맵에 대한 시각화**를 다룸

♥ 그림 5-45 CNN의 블랙박스



5.4.1 특성 맵 시각화

특성 맵(feature map), 혹은 활성화 맵

입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과

특정 입력 이미지에 대한 특성 맵을 시각화

→ 특성 맵에서 입력 특성을 감지하는 방법을 이해할 수 있도록 도움

PIL

이미지 분석 및 처리를 쉽게 할 수 있도록 도와주는 라이브러리

PIL(Python Image Library)

- 다양한 이미지 파일 형식 지원
- 이미지 처리와 그래픽 기능을 제공하는 이미지 프로세싱 라이브러리

설명 가능한 네트워크

교재에서는

- 13개의 합성곱층
- 2개의 완전연결층

으로 구성된 네트워크 생성

합성곱층과 완전연결층에서는 **ReLU 활성화 함수** 사용

inplace=True → 기존의 데이터를 연산의 결과값으로 대체한다는 의미

로그 소프트맥스(log_softmax)

신경망 말단의 결과값들을 확률 개념으로 해석하기 위해 **소프트맥스(softmax)** 함수의 결과에 **log** 값을 취한 연산

교재에서는 소프트맥스 대신 로그 소프트맥스를 사용하는 이유를 **소프트맥스가 기울기 소멸 문제(vanishing gradient problem)에 취약하기 때문**이라고 설명

모델을 생성한 후

- `model.to(device)` : CPU 혹은 GPU에 할당
- `model.eval()` : 테스트 데이터에 대한 모델 평가 용도로 사용

특성 맵 확인

특성 맵 → 합성곱층을 입력 이미지와 필터로 연산하여 얻은 결과

따라서 합성곱층의 입력과 출력을 알 수 있다면 → 특성 맵에 대한 값을 확인할 수 있음

hook / register_forward_hook

파이토치는 매 계층마다 print문을 사용하지 않더라도 **hook** 기능을 이용하여 각 계층의 활성화 함수 및 기울기 값을 확인할 수 있음

`register_forward_hook`의 목적 → **순전파 중 각 네트워크 모듈의 입력 및 출력을 가져오는 것**

코드에서 `y`, `z`와 같이 특정한 값으로 정의되지 않은 중간 변수는 계산 결과에 따라 값이 달라질 수 있는 변수

파이토치는 이러한 중간 변수에 대해서는 기울기 값을 저장하지 않음

하지만 `register_hook`을 이용하면

- 중간 변수의 기울기 값을 얻을 수 있고
- 중간 결과값들을 확인할 수 있음

cv2.resize / 보간법

`cv2.resize`

→ 이미지 크기를 변경할 때 사용

주요 파라미터

- 첫 번째 파라미터 : 변경할 이미지 파일
- 두 번째 파라미터 : 변경될 이미지 크기를 (너비, 높이)로 지정
- interpolation : 보간법

이미지 크기를 변경하면 변형된 이미지의 픽셀 값을 추정하여 값을 할당해야 함

보간법

→ 이미지상에 존재하는 픽셀 데이터에 근사 함수를 적용하여 새로운 픽셀 값을 구하는 방법

unsqueeze()

교재에서는 `unsqueeze()`를 **1차원 데이터를 생성하는 함수**라고 설명

이미지 데이터를 텐서로 변환한 후 `unsqueeze()`를 사용

교재 예)

- `dim=0` 적용 → `[3, 10, 64]`에서 `[1, 3, 10, 64]`
- `dim=1` 적용 → `[3, 10, 64]`에서 `[3, 1, 10, 64]`

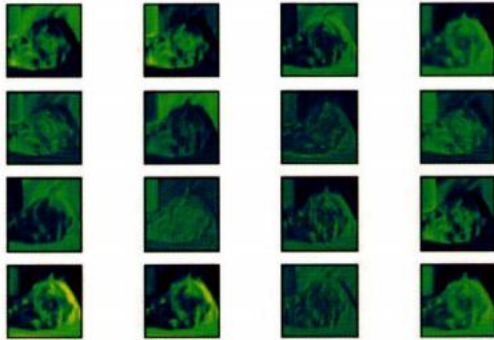
합성곱 계층별 특성 맵

0번째 Conv2d 계층

첫 번째 Conv2d 계층의 특성 맵을 확인

→ 입력층과 가까운 계층이므로 **입력 이미지의 형태가 많이 유지**

▼ 그림 5-47 첫 번째 계층에서 특성 맵

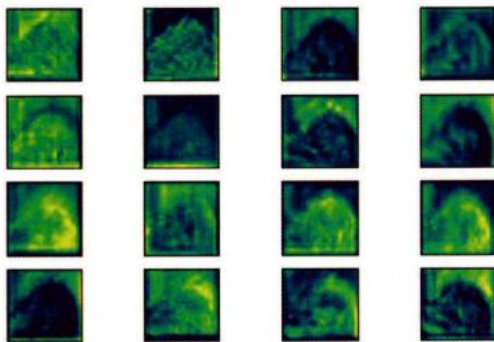


20번째 계층

20번째 특성 맵 결과

→ 기존 고양이 이미지의 형태는 찾아볼 수 없게 됨

▼ 그림 5-48 20번째 계층에서 특성 맵



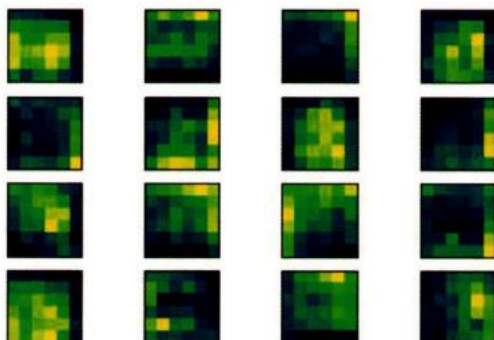
40번째 계층

40번째 계층의 특성 맵

→ 원래 입력 이미지에 대한 형태는 전혀 찾아볼 수 없음

즉, 출력층에 가까워질수록 입력 형태는 찾아볼 수 없고 이미지 특징들만 전달되는 것을 확인

▼ 그림 5-49 40번째 계층에서 특성 맵



5.5 그래프 합성곱 네트워크

그래프 합성곱 네트워크(graph convolutional network)를 이해하기 위해 앞서 그래프가 무엇인지 설명

5.5.1 그래프란

그래프

→ 방향성이 있거나(directed) 없는(undirected) 에지(edge)로 연결된 노드(nodes=vertices)의 집합

노드와 에지는 일반적으로 풀고자 하는 문제에 대한 전문가 지식이나 직관 등으로 구성

그래프의 구성 요소

노드(node, vertex)

그림에서 파란색 원

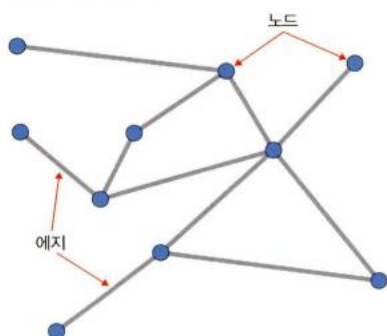
에지(edge)

두 노드를 연결한 선

즉,

- 노드 : 원소들을 의미
- 에지 : 결합 방법(single, double, triple, aromatic 등)을 의미

▼ 그림 5-50 그래프



5.5.2 그래프 신경망

그래프 신경망(Graph Neural Network, GNN)

그래프 구조에서 사용하는 신경망

그래프 데이터 표현은 두 단계로 이루어짐

1단계. 인접 행렬(adjacency matrix)

노드가 n 개일 때

→ $n \times n$ 행렬(matrix)로 표현

인접 행렬 내의 값 A_{ij}

→ i 와 j 의 관련성 여부를 만족하는 값으로 채움

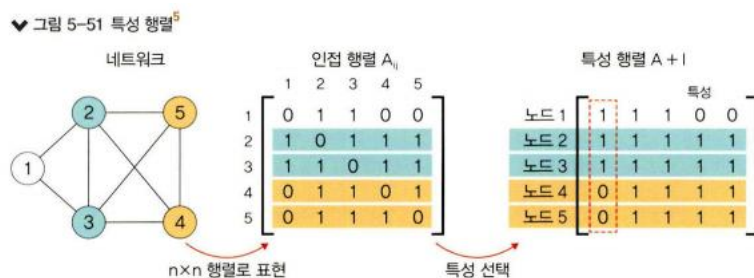
즉, 인접 행렬 과정은 컴퓨터가 이해하기 쉽게 그래프로 표현하는 과정

2단계. 특성 행렬(feature matrix)

인접 행렬만으로는 특성을 파악하기 어렵기 때문에 단위 행렬 적용

- 각 입력 데이터에서 이용할 특성을 선택
- 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미
- 각 노드에 대한 특성을 한눈에 파악하기 쉽게 표현

특성 행렬 과정을 거쳐 그래프 특성(graph feature)을 추출



5.5.3 그래프 합성곱 네트워크

그래프 합성곱 네트워크(Graph Convolutional Network, GCN)

이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘

그래프 합성곱 네트워크 구조

그래프 데이터 → 그래프 합성곱층 → 리드아웃 → 완전연결층 → 회귀 혹은 분류

리드아웃(readout)

특성 행렬을 하나의 벡터로 변환하는 함수

즉, 전체 노드의 특성 벡터에 대해 평균을 구하고 → 그래프 전체를 표현하는 하나의 벡터 생성

그래프 합성곱층(graph convolutional layer)

GCN에서 가장 중요한 부분

그래프 합성곱층을 이용하면

→ 그래프 형태의 데이터가 **행렬 형태의 데이터로 변환**

→ 딥러닝 알고리즘 적용 가능

기존 그래프 형태의 데이터에는 딥러닝 알고리즘을 적용할 수 없기 때문

GCN 활용

- SNS에서 관계 네트워크
- 학술 연구에서 인용 네트워크
- 3D Mesh

▼ 그림 5-52 그래프 합성곱 네트워크

